

AI-Controlled Android via ADB-MCP

Screen-mirroring + full device agency for an LLM through the Model Context Protocol

Prepared for	Tester Present — Specialist Automotive Solutions
Scope	Build viability, architecture, effort, and risk
Verdict	FEASIBLE — mature prior art; a screen-mirror layer is the only novel piece

Executive Summary

Giving an AI the ability to **see** a phone's screen and **act** on it as if it owned the device is not only possible — it is already a solved problem at the control layer. Multiple production-grade ADB-MCP servers exist today (phone-mcp, minhalvp/android-mcp-server, CursorTouch/Android-MCP) that expose tap, swipe, type, keypress, screenshot, UI-hierarchy extraction and raw shell to an LLM over MCP. Sending an SMS, opening an app, reading notifications, placing a call — all work now with commodity tooling. The genuinely novel component in your brief is **live screen-mirroring** (a continuous video feed rather than discrete screenshots) fused into the same MCP surface, plus a decision-loop that lets the model own the device autonomously.

Bottom line: a competent build is a **weekend-to-fortnight** effort depending on how far you push the mirroring and autonomy. The hard parts are not the ADB plumbing — they are latency, token economics on the vision loop, and the safety/authority model around an agent that can transact on a real phone.

VERDICT — Feasible. Control layer is off-the-shelf. Screen-mirroring is the one part you actually build.

1. What Already Exists (Prior Art)

The ADB-to-MCP bridge pattern is mature. You are not inventing the wheel — you are choosing a base and bolting mirroring + autonomy onto it.

Project	Lang	What it gives an LLM	Fit
hao-cyber/phone-mcp	Python	Richest high-level set: analyze-screen, send-sms, call, create-contact, Base for app, start	Best for app, start
minhalvp/android-mcp-server	Python	get_uilayout (clickable elements w/ bounds+centres), get_screenshot, Connect to adb	Connect to adb
CursorTouch/Android-MCP	Python	State/Click/Type/Swipe/Drag/Press/Notification/Shell tools, accessibility tree, screenshot	Best for app, start
landicefu/android-adb-mcp	TS	Minimalist raw-ADB abstractions	If you want TS

These prove the send-message / interact-as-owner requirement is already deliverable. The differentiator you're chasing is the *mirror*.

2. Proposed Architecture

The system splits into three planes. Only the mirror plane requires original engineering.

2.1 Control plane (solved)

- **ADB transport** — USB or TCP/IP (adb tcpip 5555) to the device or emulator.
- **Actuators** — input tap/swipe/text/keyevent, am start, service call isms or am start -a SENDTO for SMS, uiautomator dump for the view hierarchy.
- **MCP wrapper** — a server exposing each actuator as a typed tool the model calls.

2.2 Perception plane (the two options)

Approach	Mechanism	Latency	Token cost	Notes
Screenshot-on-demand (what exists now)	adb exec-out screencap → PNG per step	~0.3–1s/frame	High (full image each step)	Simple, reliable, works today
True screen-mirror (the novel build)	scrcpy H.264 stream decoded server-side; frames 100ms decoded + agent-samplable	25–100ms/frame	Controlled (sampled only when needed)	Human-observable

The pragmatic winner is a **hybrid**: run scrcpy for a continuous, human-observable mirror, and feed the model the UI-hierarchy dump as primary perception (cheap, structured, exact coordinates) with occasional decoded frames only when the hierarchy is ambiguous (canvas views, games, image content). scrcpy's own protocol already carries video + a control channel, so you can piggyback input on the same connection instead of shelling `adb input` each time — that's the latency win.

2.3 Agency plane (the interesting part)

- **Observe→Decide→Act loop** — model receives a state snapshot (hierarchy + optional frame), emits one action, receives the resulting state. This is the same pattern as computer-use agents.
- **Grounding** — prefer element-by-text/content-desc targeting over raw x,y; it survives resolution and layout changes.
- **Memory** — keep a short action/observation ring buffer so the model doesn't re-derive context every step.
- **Confirmation gates** — irreversible or costly actions (send, call, pay, delete) route through an explicit approval tool rather than firing silently.

3. Reference Implementation Sketch

Fastest credible path — fork a Python base, add a scrcpy mirror sidecar, expose both through one MCP server:

```
server.py (MCP) ■■ tool: get_state() -> uiautomator dump -> parsed elements + bounds/centres
■■ tool: get_frame() -> latest scrcpy H.264 frame -> JPEG (sampled, not streamed) ■■ tool:
tap(target) -> resolve text/desc -> scrcpy control msg (fallback: input tap) ■■ tool:
type(text) -> control channel inject ■■ tool: swipe/press/back -> control channel ■■ tool:
send_sms(no,body)-> am start -a SENDTO + confirm-gate ■■ tool: shell(cmd) -> raw adb
(guarded allowlist) mirror/ scrcpy server pushed to device, video+control socket held open
```

Everything above the mirror line exists in `minhalvp / CursorTouch` today. `get_frame()` wired to a persistent scrcpy socket is the part you write.

4. Effort & Complexity

Component	Effort	Risk
Fork ADB-MCP base, verify tap/type/SMS on your device	0.5 day	Low
Persistent scrcpy video+control sidecar	1–2 days	Medium
<code>get_frame()</code> sampling + JPEG re-encode for the model	0.5 day	Low
Element-grounded targeting (text/desc → coords)	1 day	Low
Agent loop + memory + confirmation gates	1–2 days	Medium
Hardening: allowlists, auth, wireless-ADB stability	1–2 days	Medium

Total for a solid MVP: ~5–9 working days. A rough demo that sends a message from a screenshot loop: an afternoon.

5. Constraints, Gotchas & Risk

5.1 Technical

- **SMS sending is fiddly.** Direct service call isms varies by Android version/OEM and often needs the app in foreground or default-SMS role; the reliable path is composing via intent then a scripted send-button tap.
- **Token economics.** Streaming full frames every step is expensive and slow. Hierarchy-first with sampled frames is what makes autonomy affordable.
- **Wireless ADB drops.** TCP/IP ADB is convenient but flaky under sleep/roaming — add reconnect logic.
- **Coordinate drift.** Raw x,y breaks across devices; ground on elements.
- **Latency floor.** Even with scrcpy, a full observe-decide-act cycle is model-bound (seconds), not video-bound.

5.2 Security & authority

- An agent with `shell` + input + SMS effectively owns the phone. Treat the shell tool as a loaded weapon: allowlist commands, never expose it raw to an untrusted prompt.
- Prompt-injection from on-screen content is a live threat — a malicious notification or web page can try to steer the agent. Sandbox to a test device for any untrusted workload (the CursorTouch project explicitly warns this).
- USB-debugging-enabled devices are an attack surface; the mirror/control socket must be authenticated and local-bound.

5.3 Legal / ethical

An AI acting as *if it owns* a phone — sending messages under a person's identity, transacting, reading private content — carries consent, impersonation and (in AU) potential Surveillance Devices Act and telecommunications exposure if pointed at a device or comms the operator doesn't own. Keep it scoped to

devices you control and disclose automated authorship where messages reach third parties. Same posture you already apply to owner-asset framing on your tracker work.

6. Recommendation

- **Base:** fork CursorTouch/Android-MCP (accessibility-driven, quantized screenshots, agent-oriented) or `minhalvp` for the cleanest raw base.
- **Mirror:** add a `scrcpy` sidecar; expose `get_frame()` as sampled JPEG, not a stream.
- **Perception:** hierarchy-first, frame-on-ambiguity. This is the whole trick for cheap autonomy.
- **Safety:** confirmation gates on `send/call/pay/delete`; shell allowlist; test device only for untrusted input.
- **MVP target:** "open Messages, find contact by name, compose, confirm, send" as the end-to-end proof, then generalise.

Feasible, mostly assembled from existing parts, with the screen-mirror + autonomy loop as the real engineering. The gating concern is not *can you build it* — it's the authority model around an agent that can act as the device's owner.